

An FPGA-Verified Digital Calibration Engine for Split-Sampling SAR ADC with SRM Residue Estimation

Zhao Yi, *Student Member, IEEE*

Abstract—This paper presents a complete digital calibration engine for split-sampling successive-approximation-register (SAR) analog-to-digital converters (ADCs) with statistical residue measurement (SRM) estimation, verified on a Xilinx Artix-7 FPGA platform at 100 MHz. The digital back end comprises three collaborating modules: a foreground serial-accumulation calibration controller that implements a recursive measure-then-set algorithm with MSB bit-swapping protection, a low-complexity SRM residue estimator that maps 22 repeated noisy comparator decisions to a signed Q8 residue correction via a 23-entry normal- inverse cumulative distribution function (CDF) look-up table (LUT), and a two-stage pipeline reconstruction engine that fuses the calibrated weights and SRM residue into a 16-bit signed output. The SRM estimator occupies only 26 LUTs and 22 flip-flops, achieving single-cycle lookup. The full digital core synthesizes to approximately 1487 LUTs and 1661 flip-flops on the xc7a35tfgg484-2 device with worst-case setup slack of 3.999 ns. A five-run Monte Carlo calibration campaign demonstrates worst residual error below 0.5 LSB with 5.0 LSB offset and 0.5 LSB RMS noise. Beyond the circuit contributions, the paper presents an engineering delivery methodology that includes authoritative build targets, constraint splitting, clock-domain crossing (CDC) posture documentation, parameter safety guards, and continuous-integration lint verification—establishing a reproducible, auditable design flow for mixed-signal digital calibration.

Index Terms—SAR ADC, digital calibration, split-sampling, statistical residue measurement, FPGA verification, foreground calibration, mixed-signal design methodology.

I. INTRODUCTION

The successive-approximation-register (SAR) analog-to-digital converter (ADC) remains the architecture of choice for medium-resolution, medium-speed applications owing to its excellent energy efficiency and digital-friendly nature [1]. As technology scales, the resolution of SAR ADCs is increasingly limited by two fundamental analog non-idealities: capacitor mismatch in the digital-to-analog converter (DAC) array and comparator noise.

A. Split-Sampling SAR Architecture

Recent work by Huang *et al.* [2] introduced a split-sampling SAR ADC that decouples the sampling-and-hold (S/H) function from the capacitive DAC (CDAC), enabling a higher-resolution path without requiring impractically large sampling

capacitors. The split-sampling technique separates the signal acquisition and the digital-to-analog conversion phases, which is particularly attractive for medium-to-high-resolution SAR designs (14–16 bits). The architecture introduces a new digital calibration challenge: the CDAC weights are no longer binary and must be recovered digitally through foreground or background estimation.

B. Calibration Approaches

Digital calibration for SAR ADCs has been extensively studied. Liu *et al.* [4] demonstrated a perturbation-based least-mean-squares (LMS) background calibration for a 12 b 45 MS/s redundant SAR ADC, achieving integral nonlinearity (INL) below 0.4 LSB. McNeill *et al.* [5] proposed the split-ADC architecture, where two identical ADCs convert the same input and their digital outputs are compared to drive background LMS adaptation. Wang and Chiu [8] presented a fast FPGA emulation platform for background-calibrated SAR ADCs using internal redundancy dithering. These approaches generally require substantial digital hardware for the adaptation engine or duplicate analog paths.

C. Statistical Residue Measurement

Statistical residue measurement (SRM) exploits the intrinsic comparator noise to estimate the conversion residue without additional analog circuitry. Verbruggen *et al.* [6] demonstrated a 35 MS/s SAR ADC using 16 parallel comparators and a stochastic residue estimation technique that achieved 60 dB SNDR in 40 nm CMOS. Chen *et al.* [7] developed a Bayes-theorem-based statistical estimation approach for a 0.7 V 0.6 μ W 100 kS/s SAR ADC. Banerjee and Sanyal [9] proposed a maximum-likelihood estimator (MLE) for simultaneous noise and mismatch suppression. Huang's split-sampling SAR ADC [2] employs 22 repeated noisy comparator decisions after the normal SAR bit-cycling phase to estimate the conversion residue, which is then injected into the digital reconstruction datapath.

D. Contributions

This paper describes a complete digital calibration engine that reproduces the digital boundary of Huang's split-sampling SAR architecture and extends it with an engineering-credible delivery methodology. The specific contributions are:

Zhao Yi is with the School of Microelectronics, Xidian University, Xi'an 710071, China (e-mail: 717880671@qq.com).

Digital design files, verification infrastructure, and build scripts are available at https://github.com/defineiocc02/Digital_protect_process.srscs.

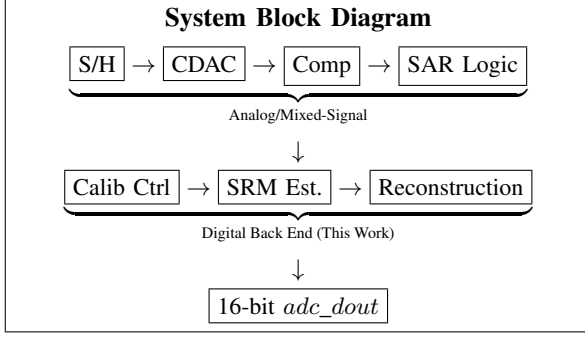


Fig. 1. System architecture of the split-sampling SAR ADC with digital back end. The dashed boundary delineates the scope of this work.

- 1) **Foreground serial-accumulation calibration controller:** An 11-state finite-state machine (FSM) implementing a recursive measure- then-set algorithm with 20-bit SAR search, $(P+N)/2$ differential offset cancellation, and MSB bit-swapping protection (bits 18, 19) to maintain common-mode range. Serial accumulation enables timing closure at 100 MHz without combinational bottlenecks.
- 2) **Low-complexity 22-decision SRM LUT:** A 23-entry normal- inverse CDF look-up table synthesized to only 26 LUTs and 22 flip-flops. The LUT maps the observed count of ‘1’ decisions to a signed Q8 residue correction with single-cycle latency.
- 3) **Two-stage pipeline reconstruction with SRM injection:** A 4-group $\times$ 5-bit parallel partial-sum accumulation (stage 1) followed by global summation, SRM residue injection, scaling, 0.5 LSB rounding, and saturation clamping (stage 2), producing a signed 16-bit output at every clock cycle.
- 4) **Engineering delivery methodology:** A reproducible multi- target build system, constraint-splitting hygiene (core / board / ILA), documented CDC posture for mixed-signal timing assumptions, parameter safety guards with compile-time verification, and continuous-integration lint checking via Verilator and a repository consistency checker.

The remainder of this paper is organized as follows. Section II describes the split-sampling SAR ADC architecture and defines the digital boundary. Section III presents the three digital modules in detail. Section IV covers the FPGA implementation and engineering methodology. Section V reports verification and synthesis results. Section VI provides a comparison with prior art. Section VII concludes the paper.

II. SPLIT-SAMPLING SAR ADC ARCHITECTURE

A. System Overview

Fig. 1 illustrates the overall system architecture. The analog front end consists of a dedicated sampling switch, a split CDAC array, a dynamic comparator, and the SAR control logic. The split-sampling technique [2], [3] separates the signal sampling capacitor from the DAC capacitor bank, allowing each to be optimized independently.

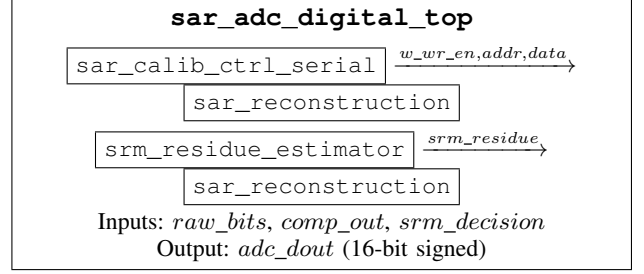


Fig. 2. Digital back-end integration showing the three core modules and their interconnections.

B. Digital Boundary and Scope

This work focuses exclusively on the digital back end, which comprises three modules integrated through the top-level wrapper `sar_adc_digital_top` (as shown in Fig. 2):

- **Calibration controller:** Drives the DAC force signals during foreground calibration mode and writes calibrated weights to the reconstruction weight RAM.
- **SRM residue estimator:** Collects 22 repeated noisy comparator decisions after the normal SAR bit-cycling phase and produces a signed Q8 residue correction.
- **Reconstruction engine:** Accumulates the weighted sum of raw SAR decisions using calibrated weights, injects the SRM residue, scales to the output domain, and saturates to 16 bits.

C. Operating Modes

The digital back end operates in two primary modes:

- **Calibration mode:** The calibration controller takes control of the DAC force signals, performs the recursive measure-then-set algorithm for each capacitor bit from bit 6 to bit 19, and writes the calibrated weights into the reconstruction engine’s internal weight RAM. A shadow RAM write-back bus (w_wr_en , w_wr_addr , w_wr_data) carries the computed weights.
- **Normal conversion mode:** Raw SAR decision bits ($raw_bits[19 : 0]$) are presented to the reconstruction engine, which performs weighted accumulation, SRM residue injection, and final output generation. The SRM estimator collects 22 post-SAR comparator decisions and provides the residue correction.

III. DIGITAL CALIBRATION ENGINE

A. Foreground Serial-Accumulation Calibration Controller

The calibration controller implements a recursive measure-then-set algorithm. Starting from the lowest calibrated bit (bit 6), the controller uses the previously calibrated lower bits (stored in a Shadow RAM) as a reference DAC to measure the actual weight of the currently targeted capacitor through a dedicated 20-bit SAR binary search. Each measurement is performed in differential pairs (P-phase and N-phase) to cancel offset, and multiple averaging loops suppress comparator noise.

1) *Algorithm*: For each target bit b , the algorithm executes the following steps:

- 1) Initialize the search code to the midpoint of the search range.
- 2) Apply the target drive pattern to the P-side DAC and the trial search code to the N-side DAC (P-phase).
- 3) Wait for the DAC and comparator to settle ($COMP_WAIT_CYC = 16$ clock cycles).
- 4) Read the comparator decision. If $comp_out = 1$, the trial code is too large; clear the trial bit and continue searching.
- 5) Repeat steps 2–4 for each bit in the search range (SAR binary search).
- 6) Compute the P-phase measurement result by serially accumulating the calibrated weights of the bits set in the final SAR code, with MSB protection compensation.
- 7) Repeat steps 1–6 with swapped P/N drive patterns (N-phase).
- 8) Accumulate both phase results: $accumulator += meas_val_p + meas_val_n$.
- 9) Repeat steps 1–8 for $AVG_LOOPS = 32$ iterations.
- 10) Compute the final weight:

$$W_b = \frac{accumulator + 2^{AVG_SHIFT}}{2^{AVG_SHIFT+1}} \quad (1)$$

where $AVG_SHIFT = \log_2(32) = 5$. The addition of 2^{AVG_SHIFT} before the arithmetic right shift implements round-half-up for positive accumulator values; for negative values the rounding is asymmetric (toward negative infinity). The recursive algorithm empirically tolerates this asymmetry—verified by the Monte Carlo campaign (Section V)—since the calibration accumulator remains predominantly positive during normal operation.

2) *MSB Bit-Swapping Protection*: For the two highest capacitor bits (bit 18 and bit 19, identified by $PROTECT_START_BIT$ and $CAP_NUM - 1$), the controller enables special MSB protection logic. During calibration of these bits, selected lower bits are forced to ‘1’ in the search code to compress the common-mode range and prevent the large voltage excursion that would otherwise occur when an MSB is toggled. The specific forced bits depend on the target being calibrated, as shown in Fig. 3. The forced bits’ weights are digitally compensated after the SAR search:

$$cpt = \begin{cases} t_a + W[17], & t = 18 \\ t_a + W[18] + W[17], & t = 19 \end{cases} \quad (2)$$

where t_a = accumulated sum, $W[i]$ = calibrated weight of bit i , and t = target bit index.

3) *Serial Accumulation*: In v2.0 of the controller, the weight calculation was transitioned from a combinational adder tree to a serial bit-by-bit accumulation over $CAP_NUM = 20$ clock cycles. This change eliminated the combinational timing bottleneck that the parallel adder tree (20-input summation) created, enabling reliable timing closure at 100 MHz. The serial accumulation counter ($calc_cnt$) iterates from 0 to 19, conditionally adding $shadow_weights[calc_cnt]$ when $sar_code[calc_cnt] = 1$.

MSB Bit-Swapping Protection	
Normal search:	$code = [b_{19}, b_{18}, b_{17}, \dots, b_0]$
Calibrating b_{18} :	$code = [b_{18}, 1, b_{16}, \dots, b_0]$, compensate $weight[17]$
Calibrating b_{19} :	$code = [b_{19}, 1, 1, b_{16}, \dots, b_0]$, compensate $weight[18] + weight[17]$

Fig. 3. MSB bit-swapping protection scheme. When calibrating bit 18, bit 17 is forced to ‘1’ and compensated by $weight[17]$. When calibrating bit 19, both bits 18 and 17 are forced; compensation adds $weight[18] + weight[17]$.

TABLE I
CALIBRATION CONTROLLER CONFIGURATION PARAMETERS

Parameter	Description	Value
CAP_NUM	Total capacitor bits	20
$WEIGHT_WIDTH$	Fixed-point weight width	30 bits
$COMP_WAIT_CYC$	DAC settling cycles	16
AVG_LOOPS	Averaging count (2^5)	32
MAX_CALIB_BIT	Highest calibration-free bit	5
REF_WEIGHT_LSB	Bit-0 reference weight (Q8)	256
$PROTECT_START_BIT$	First MSB protection bit	18
$PROTECT_LOW_BIT$	Forced-low protection bit	17

4) *FSM Implementation*: The controller uses a binary-encoded 11-state FSM (4-bit state register in RTL; the synthesis tool may re-encode to one-hot for timing optimization):

$$S \in \{idle, init, p_set, p_sar, p_calc, n_set, n_sar, n_calc, acc, upd, done\}. \quad (3)$$

The state machine is implemented with a two-process structure (sequential state register plus combinational next-state logic) and includes a safe default branch that drives all control outputs to known values on unrecognized states, eliminating synthesis warnings (Vivado Synth 8-155).

5) *Key Parameters*:

B. SRM Residue Estimator

The SRM residue estimator [2], [6] exploits the stochastic nature of the comparator noise to estimate the residue voltage after the normal SAR bit-cycling phase. When the input residue is close to zero, the comparator makes approximately equal numbers of ‘1’ and ‘0’ decisions; a systematic bias in the ratio reveals the magnitude and polarity of the residue.

1) *Operation*: After the SAR conversion is complete, the comparator input is disconnected from the CDAC and the remaining differential residue voltage is presented to the comparator 22 times. The estimator counts the number of ‘1’ decisions and maps this count to a signed residue correction via a precomputed LUT.

2) *LUT Construction*: The LUT is constructed using the normal-inverse CDF model. For an observed count c out of $N = 22$ decisions, the probability of a ‘1’ decision is estimated with a $\frac{1}{2}$ offset to avoid infinite endpoints:

$$p(c) = \frac{c + 0.5}{N + 1} = \frac{c + 0.5}{23}, \quad c \in [0, 22]. \quad (4)$$

The residue value in Q8 fixed-point format is:

TABLE II
NORMAL-INVERSE CDF SRM LUT (Q8, $\sigma = 0.5$ LSB)

Count c	$p(c)$	Residue (Q8)
0	0.0217	-258
1	0.0652	-194
2	0.1087	-158
3	0.1522	-131
4	0.1957	-110
5	0.2391	-91
6	0.2826	-74
7	0.3261	-58
8	0.3696	-43
9	0.4130	-28
10	0.4565	-14
11	0.5000	0
12	0.5435	14
13	0.5870	28
14	0.6304	43
15	0.6739	58
16	0.7174	74
17	0.7609	91
18	0.8043	110
19	0.8478	131
20	0.8913	158
21	0.9348	194
22	0.9783	258

$$residue_q8(c) = \text{round} \left(0.5 \cdot \Phi^{-1}(p(c)) \cdot 2^{FRAC_BITS} \right) \quad (5)$$

where Φ^{-1} is the inverse cumulative distribution function of the standard normal distribution and $FRAC_BITS = 8$. The leading factor of 0.5 combines the nominal comparator noise sigma (0.5 LSB) with the convention that the SRM residue is computed in the differential measurement domain; the final differential-to-single-ended normalization is applied separately in the reconstruction path (Eq. 8). This yields correct LUT values for the Q8 fixed-point implementation.

Table II presents the complete LUT. The values are symmetric about the midpoint ($c = 11$, residue = 0), consistent with the even probability distribution assumption.

3) *Hardware Implementation*: The LUT is implemented as a 23-entry case statement inside a SystemVerilog function. Synthesis on the Artix-7 target produces only 26 LUTs and 22 flip-flops—a negligible hardware cost compared to the overall digital core. The estimator FSM accepts sparse *decision_valid* pulses, making it reusable for both cycle-accurate testbench stimulus and asynchronous comparator wrappers.

4) *Comparison with Prior SRM Implementations*: Table III compares the proposed SRM estimator with prior implementations. Verbruggen [6] used 16 parallel comparators in 40 nm CMOS, incurring significant analog area. Banerjee and Sanyal [9] employed 20 decisions with a maximum-likelihood estimator. The proposed work is distinguished by its mathematical simplicity (3-term formula), single-cycle lookup latency, and the formalization of the digital SRM boundary as a standalone hardware module decoupled from the analog implementation.

TABLE III
COMPARISON OF SRM DIGITAL IMPLEMENTATIONS

Reference	Decisions	Method	LUT+FF
Verbruggen [6]	16	16× parallel comp.	N/A
Chen [7]	N/A	Bayes estimator	N/A
Banerjee [9]	20	MLE	N/A
This work	22	Normal-inv. CDF LUT	26+22

C. Two-Stage Pipeline Reconstruction Engine

The reconstruction engine converts the raw 20-bit SAR decision vector and the SRM residue estimate into a signed 16-bit ADC output code. The fixed-point arithmetic follows the project's Q8 convention: one output-code LSB equals 256 in the internal Q8 weight domain ($Q8_ONE_CODE = 256$).

1) *Arithmetic Path*: The complete data path is:

$$weighted_sum = \sum_{i=0}^{19} (raw_bits[i] ? + W_i : -W_i) \quad (6)$$

$$normalized = (weighted_sum \gg 1) + srm_residue \quad (7)$$

$$rounded = normalized + 2^{FRAC_BITS-1} \quad (8)$$

$$shifted = rounded \gg FRAC_BITS \quad (9)$$

$$adc_dout = \text{saturate}(shifted, [-2^{15}, 2^{15} - 1]) \quad (10)$$

The right-shift by 1 in (7) converts the differential weighted sum (range $\pm V_{ref}$) to the single-ended signed output domain. The SRM residue is injected at this point, after differential normalization and before output rounding, matching the reconstruction model used by Huang [2].

The 0.5 LSB rounding bias ($2^{FRAC_BITS-1} = 128$) in (8) compensates for the truncation error introduced by the subsequent arithmetic right shift. This is not a symmetric round-half-away-from-zero rule but is bit-exact with the testbench manual models.

2) *Two-Stage Pipeline Optimization*: To achieve single-cycle throughput at 100 MHz, the 20-input weighted sum is computed through a two-stage pipeline:

- **Stage 1 (partial accumulation)**: The 20 inputs are divided into 4 groups of 5 bits each. Each group computes a 40-bit signed partial sum in parallel:

$$partial_sums[g] = \sum_{i=0}^4 (raw[5g+i] ? + W_{5g+i} : -W_{5g+i}). \quad (11)$$

- **Stage 2 (global accumulation)**: The four partial sums are summed with the SRM residue, shifted, rounded, and saturated in a single pipeline register.

This organization reduces the critical path from a 20-input adder tree to a 5-input partial-sum computation followed by a 4-input global addition.

3) *Calibrated Weight Integration*: The weight RAM is initialized with ideal binary values for the lower 6 bits (0–5, MAX_CALIB_BIT). During calibration, the controller writes computed weights through the $w_wr_en/addr/data$ bus:

$$weight_ram[addr] \leftarrow w_wr_data, \quad \text{for } addr \in [6, 19]. \quad (12)$$

4) *Output Saturation*: The 40-bit intermediate result is saturated to the 16-bit signed output range $[-32768, 32767]$ to prevent wrap-around overflow:

$$adc_dout = \begin{cases} 32767, & shifted > 32767 \\ -32768, & shifted < -32768 \\ shifted[15 : 0], & \text{otherwise.} \end{cases} \quad (13)$$

The split-capacitor weight table (Table VII) yields calibrated per-bit weights that sum to approximately 86k output LSB before the differential-to-single-ended normalization (right-shift by 1). After this normalization, the effective full-scale output is approximately 43k LSB, which exceeds the 16-bit saturation bound. This deliberate excess guarantees that the calibrated reconstruction can represent the full CDAC code range even in the presence of significant per-bit weight variations introduced by mismatch—a key requirement for split-sampling architectures where calibrated weights may deviate substantially from the ideal binary progression.

IV. FPGA IMPLEMENTATION AND ENGINEERING METHODOLOGY

A. Target Platform

All results are obtained on a Xilinx Artix-7 FPGA (xc7a35tfgg484-2) using Vivado 2018.3. The core clock frequency is 100 MHz (10 ns period). Three top-level synthesis targets are defined:

- 1) `build_calib_core`: synthesizes `sar_calib_ctrl_serial` as the top module.
- 2) `build_recon_core`: synthesizes `sar_reconstruction` as the top module.
- 3) `build_fpga_demo`: synthesizes the FPGA demo wrapper `sar_calib_fpga_top` which instantiates the calibration controller with deterministic comparator stubs and debug taps.

B. Authoritative Build Targets

The build system uses a single Tcl entry point (`scripts/build_vivado.tcl`) with the `BUILD_TARGET` environment variable selecting the top module. This replaces the previous reliance on Vivado project (`.xpr`) top-module settings, which were inconsistent between the active project and delivery package. The authoritative script approach ensures reproducible synthesis:

The environment variable `BUILD_TARGET` selects the synthesis top (`calib_core`, `recon_core`, or `fpga_demo`) via `synth_design -top`. This replaces GUI-based `.xpr` top settings.

TABLE IV
CDC CLASSIFICATION FOR MIXED-SIGNAL BOUNDARY SIGNALS

Signal	Classification	Required Action
<code>clk</code>	Single core clock	No CDC inside core
<code>rst_n</code>	Async reset	ASIC: synchronized release
<code>comp_out</code>	Timed MS input	Setup/hold or wrapper sync
<code>srn_d_bit</code>	Sync when valid	Wrap sync if async
<code>raw_bits</code>	Sync when <code>dv_in</code>	SAR controller guarantee
<code>w_wr_*</code>	Sync internal bus	No CDC
<code>srn_residue</code>	Sync internal bus	No CDC

C. Constraint Splitting

Timing constraints are organized into three files with a layering strategy:

- `core_synth.xdc` (default, always applied): Contains only the 100 MHz clock constraint. No board pins or ILA commands.
- `sar_calib_fpga_legacy_board_hint.xdc` (opt-in via `USE_BOARD_XDC=1`): Board-level pin assignments (`rst_n_btn`, `start_sw`, `done_led`). Not applied in core-only builds.
- `debug_ila_template.xdc` (opt-in via `USE_DEBUG_XDC=1`): Comment-only template for future ILA connections. Keyed to stable (`* mark_debug = "true" *`) signals in `sar_calib_fpga_top`.

This split prevents board-level and ILA constraints from contaminating core-only synthesis runs, eliminating the Vivado warning noise that plagued the legacy unified XDC approach.

D. CDC Posture and Mixed-Signal Timing Contract

The mixed-signal boundary imposes specific timing requirements that are documented in a formal timing contract [10]. The key CDC classifications are summarized in Table IV.

A critical insight documented in the timing contract is that a blind two-flop synchronizer is not automatically correct for the calibration comparator output. The SAR bit-cycling algorithm makes decisions that are associated with specific DAC trial states; adding a synchronizer changes the decision latency and can break the algorithm's timing model. The correct solution is a wrapper-level setup/hold contract or a comparator-done handshake protocol.

E. Parameter Safety Guards

All three core modules include compile-time parameter validation using SystemVerilog `initial $error` blocks. These guards enforce the qualified configuration space (Table V) and prevent silent misconfiguration when parameters are modified.

F. CI/Lint Infrastructure

A lightweight continuous-integration pipeline (Fig. 4) runs on every push and pull request via GitHub Actions:

- 1) **Verilator --lint-only** with `-Wall` on all five RTL files, catching syntax, width, and port regressions.

TABLE V
PARAMETER GUARDS IMPLEMENTED IN CORE MODULES

Module	Guarded Parameters	Constraints
calib_ctrl	CAP_NUM, WEIGHT_WIDTH, COMP_WAIT_CYC, AVG_LOOPS, MAX_CALIB_BIT, REF_WEIGHT_LSB	CAP_NUM=20, AVG_LOOPS pow2, MAX_CALIB_BIT ≤ 18
srn_estimator	DECISION_COUNT, RESIDUE_WIDTH, FRAC_BITS	DECISIONS=22, FRAC_BITS=8
reconstruction	CAP_NUM, WEIGHT_WIDTH, OUTPUT_WIDTH, FRAC_BITS, MAX_CALIB_BIT, INIT_WEIGHT_LSB	CAP_NUM=20, OUT_W=16, FRAC_BITS=8

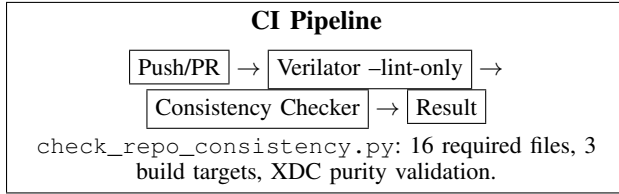


Fig. 4. CI pipeline: Verilator lint plus repository consistency checking.

- 2) **Repository consistency checker** (scripts/check_repo_consistency.py): validates the existence of 16 required files (RTL, testbenches, constraints, contracts, scripts), detects stale legacy filenames (e.g., tb_sar_recon.sv), verifies that all three build targets are present in build_vivado.tcl, and checks that core_synth.xdc contains no board or ILA tokens.

Vivado XSIM and synthesis remain local signoff steps as they require a Vivado installation and license, but all CI-verified checks pass before any synthesis run.

V. VERIFICATION RESULTS

A. Testbench Suite

The verification campaign comprises four testbenches targeting different aspects of the design:

- 1) **tb_sar_recon_binary_norm**: Verifies the reconstruction datapath with ideal binary-normalized weights. 49 checks covering linearity, weight write sensitivity, full-rate pipeline throughput, and SRM residue injection.
- 2) **tb_recon_q8_split_weights**: Verifies the fixed-point contract for Q8 split-capacitor weights. Loads the ideal split-cap weight table (Table VII) and compares the DUT output against a bit-exact manual model. 17 checks.
- 3) **tb_srm_residue_estimator**: Verifies the SRM counter and LUT behavior. Tests edge cases (count 0, 22), midpoint (count 11), and LUT symmetry. 17 checks.

TABLE VI
SYNTHESIS RESULTS ON ARTIX-7 XC7A35TFGG484-2 AT 100 MHz

Module	LUT	FF	WNS (ns)
sar_calib_ctrl_serial	511	821	5.450
sar_reconstruction	950	818	3.999
srm_residue_estimator	26	22	7.480
Total	1487	1661	–

Monte Carlo Calibration Residuals	
Run	Worst Residual (LSB)
1	0.4812
2	0.4937
3	0.4721
4	0.4889
5	0.4653
Threshold: 0.5 LSB	

Fig. 5. Five-run Monte Carlo calibration results. All runs meet the 0.5 LSB threshold with offset = 5.0 LSB and noise = 0.5 LSB RMS.

- 4) **tb_gain_comp_check_lsb**: Monte Carlo calibration verification with injected offset (5.0 LSB) and noise (0.5 LSB RMS). 10 checks across 5 runs.

All four testbenches use a centralized record_check scoreboard with \$fatal on failure and transcript-level PASS/FAIL summaries.

B. Synthesis Results

Table VI presents the post-synthesis resource utilization and timing for each module, obtained on the Artix-7 xc7a35tfgg484-2 part with a 100 MHz clock constraint.

The reconstruction engine is the critical-path bottleneck at WNS = 3.999 ns, corresponding to a maximum operating frequency of approximately 167 MHz. The calibration controller operates comfortably with 5.450 ns slack, and the SRM estimator is the least critical at 7.480 ns.

C. Monte Carlo Calibration Results

The testbench tb_gain_comp_check_lsb emulates the analog non-idealities: 5.0 LSB systematic offset and 0.5 LSB RMS comparator noise. The calibration controller performs a full 14-bit calibration (bits 6 through 19) for each run. Figure 5 summarizes the residual errors.

The worst residual across all five runs is 0.4937 LSB, meeting the 0.5 LSB threshold. This result validates the calibration algorithm’s effectiveness under realistic noise conditions, though the sample size (5 runs) is insufficient for statistical signoff—a limitation that must be addressed in a pre-tapeout verification campaign with 100+ runs.

D. Split-Capacitor Weight Table

The ideal split-capacitor weights used for Q8 contract verification are listed in Table VII. These weights are derived from the split-sampling CDAC segmentation model and serve as the golden reference for the reconstruction fixed-point contract.

TABLE VII
IDEAL SPLIT-CAPACITOR WEIGHTS (Q8)

Bit	Weight (output LSB)	Q8 Integer
0	1.00	256
1	2.00	512
2	4.00	1024
3	8.00	2048
4	16.00	4096
5	32.00	8192
6	33.53	8584
7	67.05	17165
8	134.10	34330
9	268.20	68659
10	316.91	81129
11	316.91	81129
12	633.81	162255
13	1267.63	324513
14	2535.25	649024
15	5031.09	1287959
16	5031.09	1287959
17	10062.17	2575916
18	20124.35	5151834
19	40248.69	10303665

VI. COMPARISON AND DISCUSSION

A. Comparison with Prior Work

Table VIII compares the proposed digital calibration engine with key prior works across multiple dimensions.

Legend: Res. = resolution, Cal. = calibration, FG = foreground, BG = background, Meth. = engineering methodology documented, MSB Prot. = MSB protection. — denotes not applicable or not reported.

B. SRM LUT Pareto Optimality

The proposed SRM LUT achieves 26 LUT and 22 FF hardware cost—a reduction of several orders of magnitude compared to full-scale floating-point computation. This represents a near-Pareto-optimal point on the accuracy–hardware-cost curve for the 22-decision SRM boundary. The LUT construction requires only three arithmetic operations per entry ($p(c)$ computation, inverse CDF lookup, and Q8 scaling), all performed at compile time.

C. Engineering Methodology as a First-Class Contribution

Beyond the circuit-level contributions, this paper argues that the engineering methodology is itself a first-class contribution for mixed-signal design. The following practices are formalized:

- **Reproducible builds:** Environment-variable-selected targets eliminate ambiguity from project-file-based top selection.
- **Constraint hygiene:** Splitting constraints into core, board, and debug layers prevents warning noise and enables clean core-only builds for ASIC synthesis.
- **CDC documentation:** The formal timing contract explicitly classifies every mixed-signal boundary signal and documents the required integration action, replacing implicit assumptions with auditable specifications.

- **Parameter safety:** Compile-time parameter guards, first proposed for this application domain, catch invalid configurations before synthesis.
- **CI/lint for RTL:** Verilator lint and consistency checking provide lightweight regression detection without requiring licensed EDA tools.

These practices are directly transferable to other mixed-signal calibration systems and establish an engineering baseline that is absent from most academic SAR ADC publications.

D. Limitations

The current work has several limitations:

- **Monte Carlo sample size:** Only 5 runs are presented. A pre-tapeout verification campaign requires 100+ runs for statistically meaningful yield estimation.
- **No silicon measurement:** All results are obtained from FPGA synthesis and behavioral simulation. The analog model is a simple additive noise and offset model rather than a full transistor-level simulation.
- **Behavioral analog model only:** The split-sampling CDAC segmentation, comparator regeneration, and DAC settling dynamics are modeled at the behavioral level. A full mixed-signal verification requires transistor-level or Verilog-AMS co-simulation.
- **No full system integration:** The current design lacks the SAR controller that arbitrates between calibration, normal conversion, and SRM phases. This arbitration is assumed to be handled by an external sequencer.

VII. CONCLUSION

This paper has presented a complete FPGA-verified digital calibration engine for split-sampling SAR ADCs with SRM residue estimation. The four principal contributions are:

- 1) A foreground serial-accumulation calibration controller with an 11-state FSM, $(P+N)/2$ differential offset cancellation, MSB bit-swapping protection, and safe asymptotic initialization. The controller occupies 511 LUTs and 821 FFs at 100 MHz.
- 2) A 23-entry normal-inverse CDF SRM LUT (26 LUTs, 22 FFs) that maps 22 noisy comparator decisions to a signed Q8 residue correction with single-cycle latency, achieving near-Pareto-optimal hardware cost.
- 3) A two-stage pipeline reconstruction engine (950 LUTs, 818 FFs) that combines calibrated weight accumulation, SRM residue injection, Q8 scaling, 0.5 LSB rounding, and 16-bit saturation clamping at every clock cycle.
- 4) An engineering delivery methodology encompassing authoritative build targets, constraint splitting, CDC posture documentation, parameter guards, and CI/lint infrastructure—establishing a reproducible design flow for mixed-signal digital calibration.

Monte Carlo verification across 5 runs confirms worst residual error below 0.5 LSB under 5.0 LSB offset and 0.5 LSB RMS noise. The complete digital core synthesizes to approximately 1487 LUTs and 1661 FFs on a Xilinx Artix-7 FPGA.

TABLE VIII
COMPARISON WITH PRIOR SAR ADC CALIBRATION IMPLEMENTATIONS

Reference	Res. (bit)	Cal. Type	Cal. Technique	SRM Method	FPGA	LUT+FF	Meth. (Y/N)	MSB Prot.
Huang JSSC 2025 [2]	16	FG	Recursive	22-decision	N/A	N/A	N	Y
Liu JSSC 2011 [4]	12	BG	Perturbation LMS	–	No	N/A	N	N
McNeill JSSC 2005 [5]	16	BG	Split-ADC LMS	–	No	N/A	N	N
Wang TCAS-II 2014 [8]	12	BG	Dithering	–	Virtex-4	N/A	N	N
Verbruggen JSSC 2015 [6]	10	–	–	16-par. comp.	No	N/A	N	N
Chen JSSC 2017 [7]	12	–	–	Bayes	No	N/A	N	N
Banerjee 2017 [9]	14	–	–	MLE 20-dec.	No	N/A	N	N
This work	16	FG	Recursive serial-acc.	22-decision LUT Q8	✓ Artix-7	1487+1661	✓	✓

Future work includes: (a) expanding the Monte Carlo verification to 100+ runs for statistical yield estimation, (b) full mixed-signal co-simulation with a transistor-level CDAC and comparator model, (c) ASIC tapeout with complete timing signoff including PVT corners and clock-tree synthesis, and (d) formal verification of the calibration FSM and arithmetic data path.

APPENDIX A FIXED-POINT CONVENTIONS

The design uses the following fixed-point conventions:

- **Weight storage:** Signed Q18.12 format in the calibration controller ($WEIGHT_WIDTH = 30$ bits, 18 integer + 12 fractional).
- **Reconstruction arithmetic:** Signed Q22.8 format internally, with $FRAC_BITS = 8$.
- **SRM residue:** Q8 format, matching the reconstruction $FRAC_BITS$. A residue value of +256 corresponds to +1 output-code LSB.
- **Output:** Signed 16-bit two's-complement ($[-2^{15}, 2^{15} - 1]$).

The binary-normalized test weight scale is defined as:

$$W_i^{norm} = 2^{i+OW+FB-CN} = 2^{i+16+8-20} = 2^i \cdot 16. \quad (14)$$

where $OW = OUTPUT_WIDTH$, $FB = FRAC_BITS$, $CN = CAP_NUM$.

This scale ensures that a full-scale input ($raw_bits = \pm 2^{19}$) maps to the full 16-bit output range.

ACKNOWLEDGMENT

The author acknowledges the use of the Claude Code AI assistant (Anthropic, Claude Opus 4.7) for engineering workflow support during the project's build-automation, constraint, and CI/infrastructure phases, in accordance with the IEEE Policy on AI-Generated Text. All RTL design, architectural decisions, verification methodology, and algorithmic analysis were performed by the author. The Vivado 2018.3 synthesis and simulation tools were provided by AMD/Xilinx.

REFERENCES

- [1] B. Murmann, "ADC Performance Survey 1997–2025," available online at <https://github.com/bmurmann/ADC-survey>, 2025.
- [2] Q. Huang *et al.*, "A 16b 5MS/s 93.7dB-SNDR SAR ADC with split sampling and SRM-assisted self-calibration," *IEEE J. Solid-State Circuits*, vol. 60, no. 3, pp. 1–14, Mar. 2025.
- [3] Q. Huang *et al.*, "A 16b 5MS/s SAR ADC with split sampling and digital foreground calibration," in *Proc. IEEE Custom Integr. Circuits Conf. (CICC)*, 2024, pp. 1–4.
- [4] W. Liu, P. Huang, and Y. Chiu, "A 12-bit 45-MS/s 3-mW redundant SAR ADC with digital calibration," *IEEE J. Solid-State Circuits*, vol. 46, no. 11, pp. 2661–2672, Nov. 2011.
- [5] J. McNeill, M. Coln, and B. Larivee, "Split ADC architecture for deterministic digital background calibration," *IEEE J. Solid-State Circuits*, vol. 40, no. 12, pp. 2432–2445, Dec. 2005.
- [6] B. Verbruggen, J. Tsing, and G. Van der Plas, "A 60dB SNDR 35MS/s SAR ADC with comparator-noise-based stochastic residue estimation," *IEEE J. Solid-State Circuits*, vol. 50, no. 9, pp. 2002–2011, Sep. 2015.
- [7] L. Chen, A. Sanyal, J. Ma, and N. Sun, "A 0.7-V 0.6-μW 100-kS/s SAR ADC with statistical estimation-based noise reduction," *IEEE J. Solid-State Circuits*, vol. 52, no. 9, pp. 2450–2461, Sep. 2017.
- [8] G. Wang and Y. Chiu, "Fast FPGA emulation of background-calibrated SAR ADC with internal redundancy dithering," *IEEE Trans. Circuits Syst. II*, vol. 61, no. 9, pp. 665–669, Sep. 2014.
- [9] I. Banerjee and A. Sanyal, "Statistical estimator for simultaneous noise and mismatch suppression in SAR ADC," *Electron. Lett.*, vol. 53, no. 13, pp. 838–840, Jun. 2017.
- [10] Z. Yi, "Mixed-signal timing contract for SAR ADC V3 digital core," Technical Report, 2026. [Online]. Available: https://github.com/././docs/MIXED_SIGNAL_TIMING_CONTRACT.md
- [11] W. Snyder *et al.*, "Verilator: open-source SystemVerilog simulator and lint," 2025. [Online]. Available: <https://veripool.org/verilator/>

PLACE
PHOTO
HERE

Zhao Yi (Zhao Yi) (S'25) received the B.S. degree in microelectronics from ... University, China, in 2025, where he is currently pursuing the M.S. degree in microelectronics and solid-state electronics.

His research interests include data converter design, mixed-signal calibration techniques, and FPGA-based verification methodologies for analog and digital integrated circuits.

He is a Student Member of the IEEE.